

Reviewer Pack — Free Cumulant Sum Bounded by Log-Size for Read-Twice BPs

Entry 93b47d86db79 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 03:25:23 UTC

Free Cumulant Sum Bounded by Log-Size for Read-Twice BPs

Entry ID: 93b47d86db79 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 03:25:23 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP P computing IP_2 , the sum of free cumulants $\sum_{k=1}^{\infty} \kappa_k(P) \leq \log(\text{size}(P)) + O(1)$. For random read-twice BPs over $GF(2)$, $\sum \kappa_k(P) = \Theta(\log n)$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures; their sum could distinguish read-twice BPs (with limited independence) from unrestricted BPs (with full independence). IP_2 's inherent symmetry might force linear cumulant growth.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c79489174a46a48d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def transition_matrix(P, n):
    M = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if P[i][j]:
                M[i][j] = 1
```

```

    return M

def moment_cumulant_formula(M):
    n = len(M)
    det_M = determinant(M)
    log_det_M = math.log(abs(det_M)) if det_M != 0 else -math.inf
    return log_det_M

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    P = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    T = transition_matrix(P, n)
    kappa_sum = moment_cumulant_formula(T)

    if kappa_sum == -math.inf:
        return {
            "metric_name": "kappa_sum",
            "metric_value": kappa_sum,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    size_P = n**n
    log_size_P = math.log(size_P)

    return {
        "metric_name": "kappa_sum",
        "metric_value": kappa_sum,
        "instances_tested": 1,
        "conjecture_holds": kappa_sum <= log_size_P + 1 and kappa_sum == 0(1),
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    kappa_sums = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = len(kappa_sums) / len(results)

    if support_fraction >= 0.8:
        RESULT = f"SUPPORTED mean={sum(kappa_sums)/len(kappa_sums):.2f} std={math.sqrt(sum((x - su
    else:
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        RESULT = f"FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[next(i

    print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing supporting data or counterexamples | next: Increase timeout duration and re-run tests with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	33502
2	preregistration	ollama_remote	qwen3:8b	0	0	24278
3	novelty	ollama_remote	qwen3:8b	0	0	20750
4	novelty	ollama_remote	qwen3:8b	0	0	14833
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10939
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10223
7	judge	ollama_remote	qwen3:8b	0	0	17077

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131602 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/93b47d86db79.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/93b47d86db79.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/93b47d86db79.tar.gz` (if generated)